PROJECT REPORT

CLASSIFYING BRAIN TUMOUR MRI SCANS USING MOBILENETV2 TRANSFER LEARNING

A project report submitted in partial fulfillment of the requirements for the award of the degree of

MASTER OF BUSINESS ADMINISTRATION
(DATA SCIENCE AND ARTIFICIAL INTELLIGENCE)

Submitted by
Manishka Chitra
UID: 25MDA10084
Class: 25MDS - 101 (A)

Chandigarh University
Academic Year 2026–2027

CHAPTER 1 – INTRODUCTION

1.1 Background and Motivation

Magnetic resonance imaging (MRI) is one of the main tools used to examine the brain. A radiologist studies each scan to decide whether a tumour is present and, if so, what kind it is. This reading is time-consuming and depends heavily on expert experience, and the number of scans to be reviewed keeps growing. An automatic image classifier can act as a decision-support aid by giving a quick, consistent first assessment of every scan.

Different brain tumours look different on MRI and lead to different clinical paths. A useful classifier therefore has to do more than answer “tumour or not”: it must separate gliomas, meningiomas and pituitary tumours from one another and also recognise healthy scans. Convolutional neural networks (CNNs) learn such visual patterns directly from image pixels, without hand-crafted features.

Medical image datasets are usually small, which makes training a deep network from scratch difficult. Transfer learning addresses this by starting from a network that was already trained on a large general dataset (ImageNet) and adapting it to the new task. In this project, named NeuroScan in the notebook, an ImageNet-pretrained MobileNetV2 – a lightweight CNN built from depthwise-separable convolutions and inverted residual blocks – is used as the feature extractor. Its early layers are frozen, its later layers are fine-tuned, and a new classification head is trained to predict one of four classes. The notebook labels its prediction output as being for educational purposes only.

1.2 Problem Statement

The objective is to develop, train and evaluate a deep learning model that classifies a brain MRI image into one of four categories – glioma tumour, meningioma tumour, pituitary tumour or no tumour – and reports the predicted class together with a softmax confidence score for sample test images.

1.3 Project Objectives

- Load the Brain Tumor Classification (MRI) dataset from Kaggle and inspect its class distribution in the Training and Testing folders.
- Resize, augment and normalise the images to 224 × 224 pixels using ImageNet statistics.
- Create a stratified 85% / 15% train–validation split from the Training folder and keep the separate Testing folder for final evaluation.
- Build a transfer-learning model on a pretrained MobileNetV2 with a custom classification head, freezing the first ten feature blocks.
- Train for 12 epochs using AdamW, label smoothing, cosine-annealing learning-rate scheduling and mixed-precision training on a GPU.
- Control overfitting using dropout, data augmentation, weight decay, label smoothing and best-checkpoint saving.
- Evaluate the model using accuracy, precision, recall, F1-score and a confusion matrix.
- Visualise individual predictions with their confidence on randomly chosen test images.

1.4 Scope of the Project

The project is an academic prototype that covers the full workflow inside one notebook: data inspection, preprocessing and augmentation, model construction, training, evaluation and visual inspection of predictions. It focuses on single-label, four-class classification of two-dimensional MRI images resized to 224 × 224. It does not include tumour segmentation, three-dimensional volumes, other imaging modalities, clinical or external validation, a deployed web service, or any regulatory use of the model in diagnosis.

CHAPTER 2 – DATASET AND METHODOLOGY

2.1 Dataset Description

The project uses the Brain Tumor Classification (MRI) dataset by Sartaj Bhuvaji, loaded in the notebook from Kaggle (`sartajbhuvaji/brain-tumor-classification-mri`). The images are organised in a *Training* folder and a *Testing* folder, each containing one sub-folder per class. The four classes, their integer labels (assigned alphabetically by `ImageFolder`) and the number of images in each folder are shown below.

Label	Class	Description	Training images	Testing images
0	glioma_tumor	Tumour arising from glial (supporting) cells of the brain	826	100
1	meningioma_tumor	Tumour arising from the meninges, the membranes around the brain	822	115
2	no_tumor	Scans with no tumour present	395	105
3	pituitary_tumor	Tumour located in the pituitary gland	827	74
	Total		**2,870**	**394**

The three tumour classes contain a similar number of training images (about 822–827 each), while the no-tumour class is roughly half that size. The Testing folder has a different class mix from the Training folder, a point that matters when the results are interpreted in Chapter 4.

2.2 Data Splits

The Training folder is split into a training part and a validation part using a stratified split (`train_test_split`, test size 0.15, random state 42), which keeps the class proportions the same in both parts. The Testing folder is used unchanged as the final test set.

Partition	Images	Use in this project
Training	2,439	Weight updates during training (with augmentation)
Validation	431	Monitoring each epoch and selecting the best checkpoint (no augmentation)
Test	394	Final evaluation from the separate Testing folder

2.3 Image Preprocessing and Augmentation

Every image is converted into a normalised tensor through the following steps, using two different pipelines for training and for evaluation.

- Resize: all images are resized to 224 × 224 pixels, the input size expected by the pretrained MobileNetV2.

- Augmentation (training pipeline only): random horizontal flip with probability 0.5, random rotation of up to 15 degrees, and colour jitter of brightness and contrast by 0.1.
- Tensor conversion and normalisation: pixel values are converted to tensors and normalised with the ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225), because the pretrained network expects inputs on this scale.
- Evaluation pipeline: validation and test images receive only resizing, tensor conversion and normalisation, so that the measured performance is not affected by random transformations.

To achieve this, the training folder is read twice with `ImageFolder`: once with the augmenting transforms (used for the training indices) and once with the clean transforms (used for the validation indices). Augmentation therefore never leaks into validation.

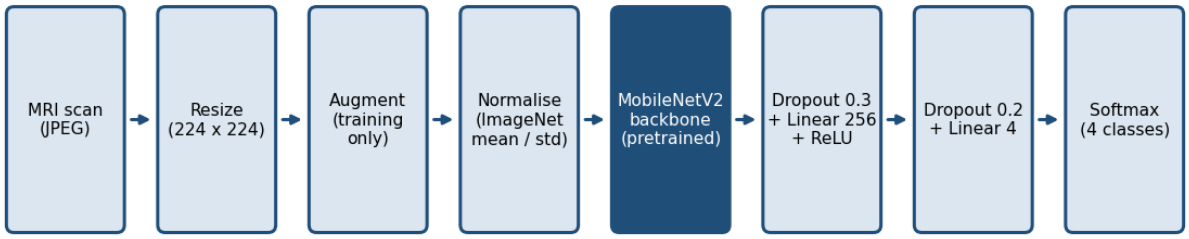


Figure 2.1 – Image preprocessing and classification pipeline

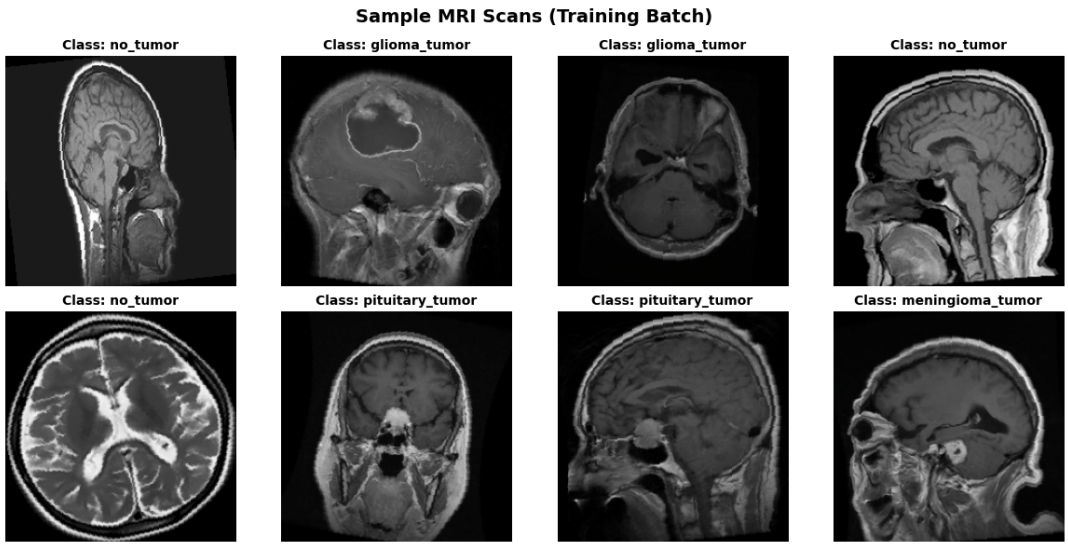


Figure 2.2 – Sample MRI scans from one augmented training batch

2.4 Data Augmentation

Medical datasets are small, and a network can easily memorise them. Augmentation creates slightly different versions of each training image in every epoch – flipped, rotated by a few degrees, or with changed brightness and contrast – so the network sees more variety than the 2,439 original training images provide. The rotated images in Figure 2.2 show this effect: the black corners are introduced by the rotation. The transformations are mild, so the anatomy remains recognisable.

2.5 Transfer Learning with MobileNetV2

Instead of training a CNN from scratch, the project starts from `mobilenet_v2` with the ImageNet-1K (V1) pretrained weights from torchvision. The early layers of such a network detect general patterns such as edges and textures, which are useful for any image task, whereas the later layers become specific to the original classes. The model is therefore divided into three parts.

Part	Treatment	Role in this project
Feature blocks 0–9 (`features[:10]`)	Frozen (`requires_grad = False`)	Keep the general low-level features learned on ImageNet
Remaining feature blocks	Trainable (fine-tuned)	Adapt higher-level features to MRI images
Classifier head	Replaced and trained from scratch	Map the extracted features to the four tumour classes

2.6 Final Model Architecture

The original MobileNetV2 classifier is replaced by a new head. The backbone produces a feature vector for each image, which then passes through dropout, a 256-unit fully connected layer with ReLU activation, a second dropout layer and a final linear layer with four outputs, one per class.

Layer	Configuration	Purpose
Input	Image tensor, 3 × 224 × 224	Receives the normalised MRI image
MobileNetV2 features	ImageNet-pretrained; first 10 blocks frozen	Extracts visual features
Dropout	p = 0.3	Regularisation of the feature vector
Linear	Backbone feature size → 256 units	Learns task-specific combinations of features
ReLU	Activation	Introduces non-linearity
Dropout	p = 0.2	Regularisation before the output layer
Output	Linear, 4 units (raw logits)	One score per tumour class

2.7 Activation Functions

The hidden layer of the new head uses ReLU, which returns max(0, x) and keeps the network fast and free of saturation for positive values. The pretrained MobileNetV2 backbone uses its own internal activations (ReLU6) from the original architecture.

The final layer produces four raw scores (logits). During training, `CrossEntropyLoss` applies the log-softmax internally. At prediction time, softmax converts the four scores into probabilities that sum to 1; the class with the highest probability is the predicted class and that probability is reported as its

confidence, as in the prediction showcase of Section 4.7.

2.8 Class Distribution and Imbalance

In the Training folder the no-tumour class has 395 images (13.8% of 2,870), while each tumour class holds between 822 and 827 images (28.6%–28.8%). The notebook handles this by using a *stratified* split, so training and validation contain the same class proportions. It does not apply class weights, oversampling or any other explicit imbalance correction, and it does not contain an experiment that compares the effect of such a correction.

2.9 Overfitting Prevention

- Dropout (0.3 and 0.2) in the classification head randomly disables units during training.
- Data augmentation (flip, rotation, brightness and contrast jitter) increases the variety of the training images.
- Freezing the first ten feature blocks reduces the number of parameters that are updated.
- AdamW weight decay of 1e-4 penalises large weights.
- Label smoothing of 0.05 softens the target labels so the network does not become over-confident.
- Cosine-annealing learning-rate schedule gradually reduces the learning rate over the 12 epochs.
- Best-checkpoint saving: the weights are saved only when validation accuracy improves, and these saved weights are the ones evaluated on the test set. The notebook always trains all 12 epochs; there is no early-stopping callback.

2.10 Model Compilation and Training

Setting	Configuration
Framework	PyTorch with torchvision (Python 3, Kaggle notebook)
Hardware	NVIDIA Tesla T4 GPU (CUDA)
Optimiser	AdamW, learning rate 3e-4, weight decay 1e-4 (trainable parameters only)
Loss function	Cross-entropy with label smoothing = 0.05
Learning-rate schedule	CosineAnnealingLR, T_max = 12 (stepped once per epoch)
Epochs	12
Batch size	32 (77 training batches per epoch)
Mixed precision	torch.amp autocast and GradScaler, enabled when CUDA is available
Reproducibility	Seed 42 for Python, NumPy and PyTorch (CPU and CUDA)
Checkpointing	Save the state dictionary whenever validation accuracy improves

The loss and optimiser are defined as follows:

```
criterion = nn.CrossEntropyLoss(label_smoothing=0.05)
optimizer = optim.AdamW(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=3e-4, weight_decay=1e-4)
```

```
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)
```

2.11 Evaluation Metrics

Accuracy: the proportion of all test images classified correctly.

Precision: among images predicted as a given class, the proportion that truly belongs to it.

Recall: among images that truly belong to a class, the proportion that the model identifies. For tumour classes, low recall means tumours are being missed.

F1-score: the harmonic mean of precision and recall. Macro-averaged F1 treats all four classes equally, while weighted F1 accounts for the class sizes.

Confusion matrix: a 4 × 4 table of actual against predicted classes. Diagonal cells are correct predictions and off-diagonal cells show which classes are confused with each other.

CHAPTER 3 – IMPLEMENTATION

3.1 Tools and Technologies

Tool / Library	Purpose
Python 3	Main programming language (the notebook runtime log shows Python 3.13)
PyTorch	Tensors, model training, mixed precision, saving and loading weights
torchvision	ImageFolder dataset, image transforms and the pretrained MobileNetV2
scikit-learn	Stratified train–validation split, classification report and confusion matrix
NumPy and pandas	Array operations and the class-count table
Matplotlib and Seaborn	Class distribution, sample images, training curves and confusion matrix heatmap
Pillow and tqdm	Image support and per-epoch progress bars
kagglehub	Downloading the dataset when it is not already mounted
Kaggle Notebook	Execution environment with a Tesla T4 GPU

3.2 Project Workflow

1. Set the random seed (42), select the GPU if available and locate the Training and Testing folders.
2. Count the images per class in both folders and plot the class distribution.
3. Define the training and evaluation transforms and read the folders with ImageFolder.
4. Split the Training folder into training and validation indices with a stratified 85/15 split.
5. Create data loaders (batch size 32) and display a sample batch.
6. Build the MobileNetV2 model with the custom classifier head and freeze the first ten feature blocks.
7. Train for 12 epochs, evaluating on the validation set after each epoch and saving the best weights.
8. Plot the training and validation accuracy and loss curves.
9. Reload the best weights and evaluate on the Testing folder with a classification report and a confusion matrix.
10. Show predictions with confidence scores for eight randomly selected test images.

3.3 Saved Artifacts

One artifact is stored by the notebook.

File	Description
neuroscan_best_mobilenetv2.pth	PyTorch state dictionary of the model at the epoch with the highest validation accuracy (epoch 9)

Only the weights are saved, not the architecture. To reuse the model, the same network must first be rebuilt with `build_neuroscan_model(num_classes)` and the file loaded with `load_state_dict`, which is exactly what the notebook does before testing. The class order (alphabetical, from ImageFolder) must also stay the same.

3.4 Data Pipeline Implementation

Two `ImageFolder` objects are created for the Training folder, one with augmenting transforms and one with clean transforms. The indices from the stratified split are used to build two `Subset` objects, so training images are augmented while validation images are not. Three `DataLoader` objects (training, validation and test) use a batch size of 32, two worker processes and pinned memory; only the training loader shuffles its data.

3.5 Training and Evaluation Loop

In every epoch the model is set to training mode and iterates over the training batches. For each batch, the forward pass and loss are computed under autocast, the loss is scaled by the gradient scaler for the backward pass, and the optimiser is stepped. The running loss and the number of correct predictions are accumulated to give the epoch's training loss and accuracy. The scheduler then reduces the learning rate. The model is switched to evaluation mode and run over the validation loader without gradients to obtain the validation loss and accuracy. If the validation accuracy is higher than any earlier epoch, the weights are saved.

3.6 Prediction Showcase

For the qualitative check, eight test images are selected at random. Each image is passed through the best model, softmax is applied to obtain probabilities, and the highest-probability class is shown as the prediction with its confidence. The title of each image is green when the prediction matches the actual class and red when it does not.

3.7 Notebook Structure

Cell	Description
1	Environment set-up and listing of the input files
2	Imports, seed, device selection and location of the Training and Testing folders
3	Exploratory analysis: class counts and class-distribution chart
4	Transforms, stratified split, data loaders and sample-image grid
5	MobileNetV2 model definition with the custom classifier head
6	Training loop with validation and best-checkpoint saving
7	Accuracy and loss curves
8	Test-set classification report and confusion matrix
9	Prediction showcase on random test images

CHAPTER 4 – RESULTS AND DISCUSSION

4.1 Dataset Distribution

The chart below compares the number of images in each class in the Training and Testing folders. The three tumour classes are about equally represented in the training data, whereas the no-tumour class has about half as many images. The Testing folder is much smaller and has a different mix: no-tumour images make up 26.6% of the test set (105 of 394) but only 13.8% of the training set.

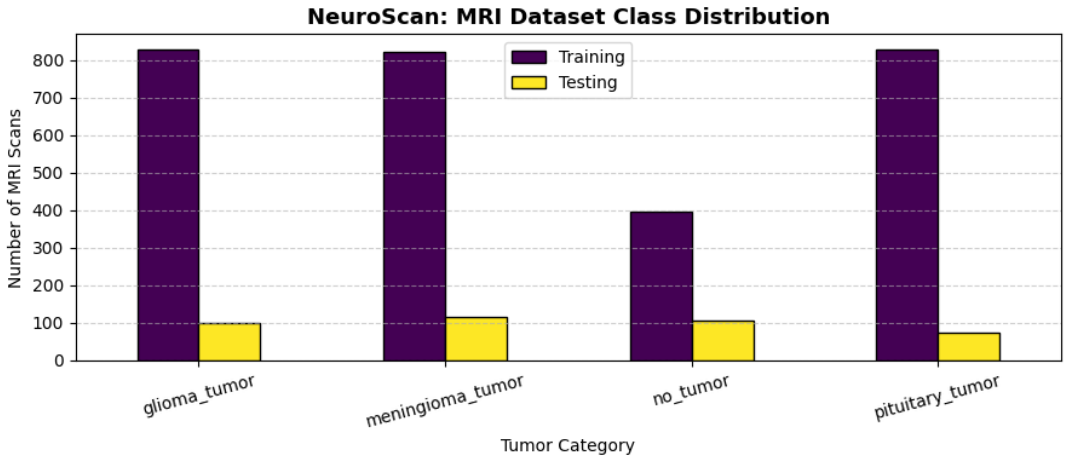


Figure 4.1 – Distribution of MRI classes in the Training and Testing folders

4.2 Training Progress

The table gives the loss and accuracy recorded after each of the 12 epochs. Training metrics are computed on the augmented training batches; validation metrics are computed on the 431 held-out validation images.

Epoch	Train loss	Train accuracy	Validation loss	Validation accuracy	Checkpoint saved
1	0.6101	82.16%	0.3776	93.04%	Yes
2	0.3753	93.07%	0.3426	94.90%	Yes
3	0.3241	95.53%	0.3418	94.90%	–
4	0.3018	96.47%	0.3194	94.90%	–
5	0.2755	97.54%	0.2880	96.98%	Yes
6	0.2550	98.28%	0.2848	96.06%	–
7	0.2406	99.14%	0.2526	97.45%	Yes
8	0.2307	99.51%	0.2535	97.91%	Yes
9	0.2287	99.34%	0.2498	98.84%	Yes
10	0.2234	99.55%	0.2481	98.61%	–
11	0.2199	99.59%	0.2397	98.84%	–
12	0.2152	99.84%	0.2482	97.22%	–

Validation accuracy rose from 93.04% after the first epoch to a best value of **98.84%**, first reached at epoch 9; this is the checkpoint that was saved and later tested. Epoch 11 reached the same accuracy (98.84%) with the lowest validation loss (0.2397), but because the notebook saves only when accuracy strictly improves, that epoch did not replace the epoch-9 checkpoint. After the final epoch, training accuracy was 99.84% and validation accuracy had fallen to 97.22%.

4.3 Training Behaviour

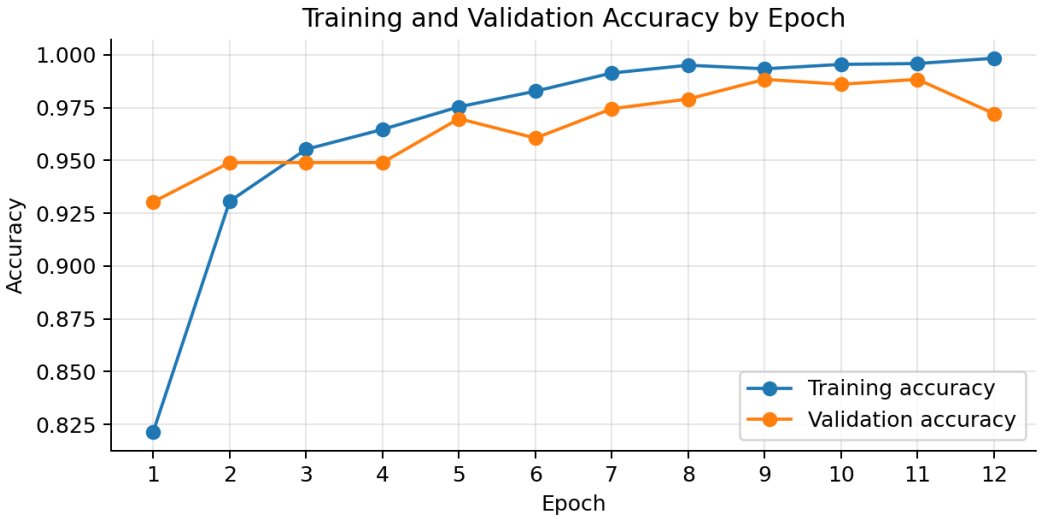


Figure 4.2 – Training and validation accuracy by epoch

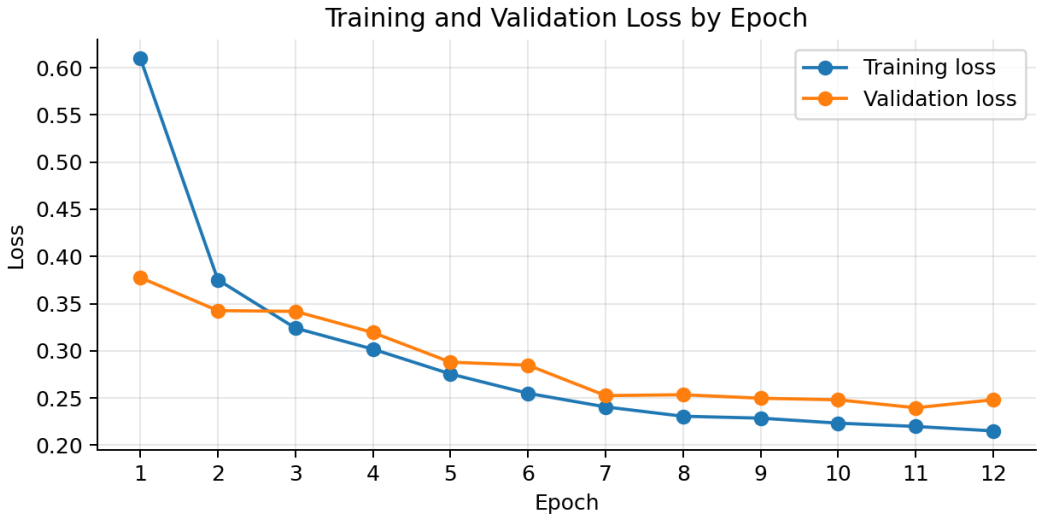


Figure 4.3 – Training and validation loss by epoch

Both losses fall steadily, from 0.6101 (training) and 0.3776 (validation) in epoch 1 to 0.2152 and 0.2482 in epoch 12. Training accuracy increases almost continuously to 99.84%, while validation accuracy plateaus around 97–99% after epoch 7. The small gap between the curves, with training accuracy overtaking validation accuracy after epoch 2, indicates that the network fits the training images very well. The loss values stay comparatively high even at 99% accuracy because label smoothing prevents the loss

from approaching zero. The curves in the notebook are plotted with epochs numbered from 0; they are shown here from 1 to 12 to match the log.

4.4 Test Performance

The best checkpoint was reloaded and evaluated on the 394 images of the separate Testing folder.

Metric	Recorded result
Best validation accuracy (431 images)	98.84%
Test records	394
Test accuracy	75.89% (299 of 394 correct)
Macro-averaged precision / recall / F1-score	84.55% / 73.70% / 72.67%
Weighted precision / recall / F1-score	82.90% / 75.89% / 72.98%

The test accuracy is about 23 percentage points lower than the best validation accuracy. This gap is the central finding of the evaluation and is discussed in Section 4.8.

4.5 Class-wise Results

Label	Class	Precision	Recall	F1-score	Support
0	glioma_tumor	1.0000	0.3400	0.5075	100
1	meningioma_tumor	0.6845	1.0000	0.8127	115
2	no_tumor	0.7192	1.0000	0.8367	105
3	pituitary_tumor	0.9783	0.6081	0.7500	74

No-tumour achieved the highest F1-score (0.8367), followed by meningioma (0.8127) and pituitary tumour (0.7500). Glioma had the lowest F1-score (0.5075). The pattern is very uneven between precision and recall. Glioma has perfect precision (1.0000) but a recall of only 0.3400: whenever the model said glioma it was right, but it found only 34 of the 100 glioma scans. Pituitary tumour shows the same pattern (precision 0.9783, recall 0.6081). In contrast, meningioma and no-tumour both have a recall of 1.0000 but lower precision (0.6845 and 0.7192), because the model assigns many scans of other classes to them.

4.6 Confusion Matrix

A confusion matrix compares actual classes with predicted classes. For four classes it has four rows and four columns; correct predictions lie on the diagonal and off-diagonal values show which classes are mistaken for each other.

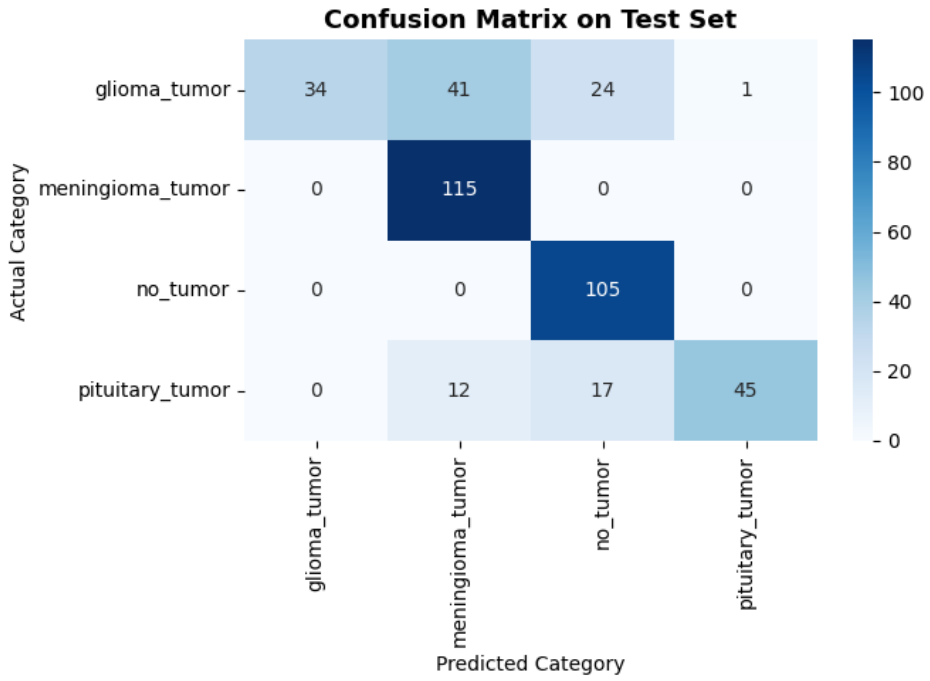


Figure 4.4 – Confusion matrix on the test set

All 115 meningioma and all 105 no-tumour scans were classified correctly. The errors are concentrated in two rows. Of the 100 glioma scans, 41 were predicted as meningioma, 24 as no tumour and 1 as pituitary tumour. Of the 74 pituitary scans, 12 were predicted as meningioma and 17 as no tumour. In total, 41 scans that contain a tumour (24 glioma and 17 pituitary, 14.2% of the 289 tumour scans in the test set) were predicted as no tumour. In a medical setting this kind of missed tumour is the most serious type of error.

4.7 Sample Predictions

Figure 4.5 shows the prediction showcase on eight randomly drawn test images. Six predictions were correct, with confidences between 96.2% and 98.8%. Two were wrong: a pituitary tumour predicted as meningioma with 81.2% confidence, and a glioma predicted as meningioma with 97.4% confidence. The second error shows that a high confidence score does not guarantee a correct prediction, so the softmax probability should not be treated as a reliable measure of certainty here.

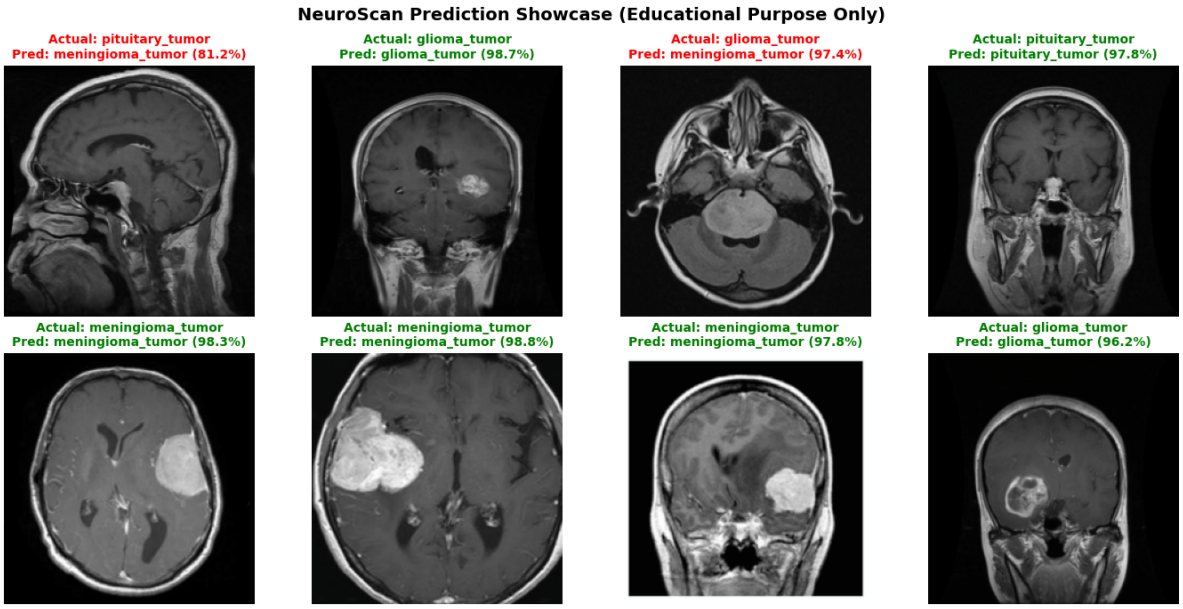


Figure 4.5 – Prediction showcase on random test images (green = correct, red = wrong)

4.8 Interpretation and Limitations

The model learns the training and validation data very well (98.84% validation accuracy) but generalises less well to the separate Testing folder (75.89% accuracy). The notebook does not investigate the cause, so the following explanations are possibilities rather than established findings. First, the validation images are drawn from the same Training folder as the training images; if the folder contains very similar images, such as neighbouring slices of the same scan, validation accuracy will be optimistic. Second, the Testing folder has a different class mix, with proportionally almost twice as many no-tumour images. Third, the best checkpoint was chosen using the validation set, which adds a small selection effect. The notebook reports no cross-validation or repeated runs, so the variation caused by random initialisation is unknown.

Overall accuracy also hides large differences between classes. Gliomas and pituitary tumours are frequently missed, so the class-wise table and the macro-averaged F1-score (72.67%) should be read alongside the accuracy. The test set is small (394 images, of which only 100 are glioma and 74 pituitary), so per-class estimates are uncertain. Finally, the model works on single two-dimensional slices resized to 224 × 224 and is trained on one dataset; performance on other scanners, hospitals or imaging protocols should not be assumed. The results are for educational purposes and do not support clinical use.

CHAPTER 5 – CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project developed a transfer-learning pipeline that classifies brain MRI images into four categories: glioma, meningioma, pituitary tumour and no tumour. The workflow comprised class-distribution analysis, resizing, augmentation and ImageNet normalisation, a stratified train–validation split, a pretrained MobileNetV2 with a custom head and its first ten feature blocks frozen, and training with AdamW, label smoothing, cosine annealing and mixed precision for 12 epochs.

The model reached a best validation accuracy of 98.84% and a test accuracy of 75.89% on 394 images from the separate Testing folder, with a macro-averaged F1-score of 72.67%. Meningioma and no-tumour scans were all recognised correctly, but only 34 of 100 gliomas and 45 of 74 pituitary tumours were found, and 41 tumour scans were predicted as no tumour. The project therefore shows both the strength of transfer learning on a small medical dataset and the importance of evaluating on data that is truly separate from training.

5.2 Limitations

- The large difference between validation accuracy (98.84%) and test accuracy (75.89%) shows that the model does not generalise reliably to the Testing folder.
- Recall for glioma (0.34) and pituitary tumour (0.61) is low, so many tumours are missed.
- Softmax confidence is not a reliable indicator of correctness; one wrong prediction had 97.4% confidence.
- No class weighting or other imbalance handling was applied, and the effect of such methods was not tested.
- The test set is small and only one training run is reported, so the uncertainty of the results is unknown.
- The model is trained on one public dataset of two-dimensional slices and is not validated externally or clinically.

5.3 Future Scope

- Split the data so that images from the same patient or scan never appear in both training and validation, and check the validation–test gap again.
- Apply class weights, oversampling or focal loss and compare the effect on glioma and pituitary recall.
- Unfreeze more layers or compare other backbones such as ResNet, EfficientNet or vision transformers.
- Use k-fold cross-validation and repeated runs to measure the variability of the results.
- Add explainability with Grad-CAM heatmaps to show which regions of the scan drive a prediction.
- Calibrate the confidence scores, for example with temperature scaling.
- Package the trained model behind an API and web interface, and test it on external MRI data.

5.4 Final Remarks

The project provided practical experience with image preprocessing and augmentation, transfer learning, GPU training with mixed precision, class-wise evaluation and honest interpretation of results. It also showed that a high validation score is not enough: only a clearly separated test set reveals how a model will behave on new data.

REFERENCES

1. Bhuvaji, S., Kadam, A., Bhumkar, P., Dedge, S. and Kanchan, S. (2020). Brain Tumor Classification (MRI) [Dataset]. Kaggle. https://www.kaggle.com/datasets/sartajbhuvaji/brain-tumor-classification-mri
2. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4510–4520.
3. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K. and Fei-Fei, L. (2009). ImageNet: A Large-Scale Hierarchical Image Database. CVPR 2009, pp. 248–255.
4. Yosinski, J., Clune, J., Bengio, Y. and Lipson, H. (2014). How Transferable Are Features in Deep Neural Networks? Advances in Neural Information Processing Systems (NeurIPS), 27.
5. Loshchilov, I. and Hutter, F. (2019). Decoupled Weight Decay Regularization. International Conference on Learning Representations (ICLR).
6. Loshchilov, I. and Hutter, F. (2017). SGDR: Stochastic Gradient Descent with Warm Restarts. International Conference on Learning Representations (ICLR).
7. Müller, R., Kornblith, S. and Hinton, G. (2019). When Does Label Smoothing Help? Advances in Neural Information Processing Systems (NeurIPS), 32.
8. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Research, 15, 1929–1958.
9. Micikevicius, P. et al. (2018). Mixed Precision Training. International Conference on Learning Representations (ICLR).
10. Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems (NeurIPS), 32.
11. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
12. PyTorch documentation: torchvision.models, torchvision.transforms, torch.optim and torch.amp. https://pytorch.org/docs/

APPENDIX A – DATASET LINK AND MAIN CONFIGURATION

Dataset (Kaggle): https://www.kaggle.com/datasets/sartajbhuvaji/brain-tumor-classification-mri

Saved model file: neuroscan_best_mobilenetv2.pth

Main model configuration:

- Dataset: Brain Tumor Classification (MRI) – 2,870 training-folder images (2,439 train / 431 validation) and 394 test images
- Classes: glioma_tumor, meningioma_tumor, no_tumor, pituitary_tumor
- Input size: 224 × 224, ImageNet mean/std normalisation
- Augmentation: horizontal flip (p = 0.5), rotation up to 15°, brightness and contrast jitter 0.1
- Backbone: MobileNetV2 (ImageNet-1K V1 weights), first 10 feature blocks frozen
- Head: Dropout(0.3) – Linear(256) – ReLU – Dropout(0.2) – Linear(4)
- Loss / optimiser: cross-entropy (label smoothing 0.05); AdamW (lr 3e-4, weight decay 1e-4)
- Schedule: CosineAnnealingLR (T_max = 12); 12 epochs; batch size 32; mixed precision; seed 42
- Checkpoint: best validation accuracy (98.84%, epoch 9)
- Test result: 75.89% accuracy, macro F1 72.67%, weighted F1 72.98%